

NETTOPOLOGY SUITE (NTS) COMPLETE IMPLEMENTATION GUIDE

For RePlot Land Readjustment Application

TABLE OF CONTENTS

1. [What is NetTopology Suite? \(Layman's Terms\)](#)
 2. [Why NTS for Your RePlot Application?](#)
 3. [Core Architecture Integration](#)
 4. [Complete NTS Class Reference](#)
 5. [Database Integration with Entity Framework](#)
 6. [Parcel Geometry Operations](#)
 7. [Service Layer Implementation](#)
 8. [Practical Code Examples](#)
 9. [Performance Optimization](#)
 10. [Common Pitfalls and Solutions](#)
-

1. WHAT IS NETTOPOLOGY SUITE?

Simple Explanation

Imagine you're working with land parcels on paper. You can:

- Draw boundaries (lines and polygons)
- Measure areas and distances
- Check if two parcels touch each other
- Combine parcels into larger plots
- Split parcels into smaller pieces
- Find if a point is inside a parcel

NetTopology Suite (NTS) is like a **super-smart digital ruler and calculator** that does all these operations on your computer, but with extreme precision (to millimeter accuracy).

Real-World Analogy

Think of NTS as a **CAD engineer's toolbox**:

Physical Tool	NTS Equivalent	What It Does
Ruler	LineString	Measures distances and creates boundaries
Compass	Point	Marks exact locations (corners, centers)
Area Calculator	Polygon.Area	Calculates parcel areas
Tracing Paper	Intersection()	Finds where two parcels overlap
Scissors	Difference()	Cuts out one shape from another
Glue	Union()	Merges multiple parcels into one

Key Concept: Geometry = Math + Geography

NTS represents everything as **coordinates** (X, Y values):

Your Parcel Corner → Point(535123.45, 3012456.78)

Your Parcel Boundary → LineString connecting multiple Points

Your Complete Parcel → Polygon (closed LineString with area)

2. WHY NTS FOR REPLOT?

Problems It Solves

Your RePlot Need	Without NTS	With NTS
Accuracy	Manual calculations prone to errors	Sub-millimeter precision

Your RePlot Need	Without NTS	With NTS
Topology Validation	Can't detect self-intersecting parcels	Automatic validation
Complex Operations	Merging parcels requires manual calculation	Single .Union() call
Database Storage	Store coordinates as text/JSON	Industry-standard WKB format
GIS Compatibility	Custom format nobody understands	Works with ArcGIS, QGIS, AutoCAD
Adjacency Detection	Loop through all parcels manually	Fast R-Tree spatial index

Industry Standard

NTS implements the **OGC Simple Features Specification** - the same standard used by:

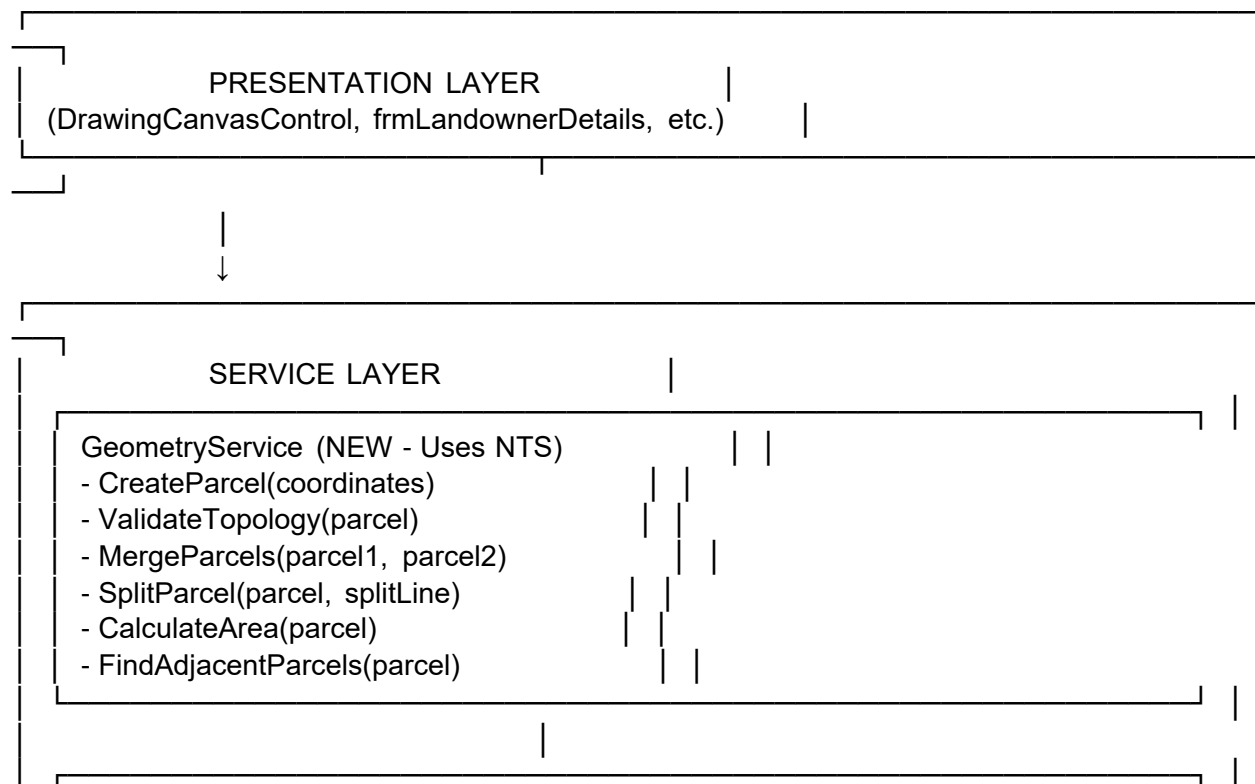
- AutoCAD Civil 3D
- ESRI ArcGIS
- QGIS
- PostGIS (PostgreSQL)
- SQL Server Spatial
- Oracle Spatial

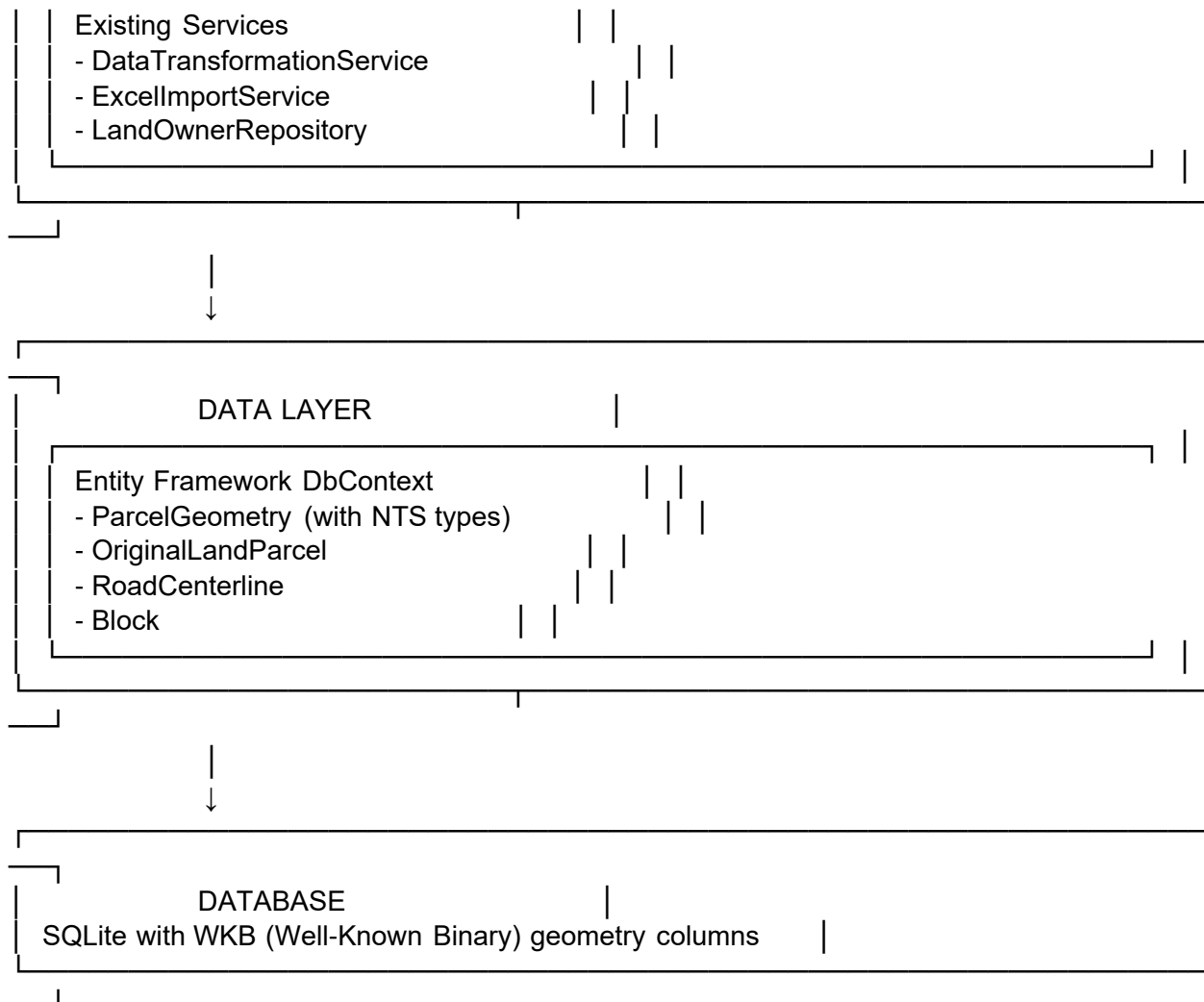
This means: Data created in your RePlot app can be exported/imported to any professional GIS/CAD software.

3. CORE ARCHITECTURE INTEGRATION

Your Current RePlot Architecture

Based on your project structure, here's how NTS fits in:





NTS Integration Points

// 1. SHAPE MODELS (Your existing IShape interface)

public interface IShape

{

Guid Id { get; }

string LayerName { get; set; }

// ADD THIS:

NetTopologySuite.Geometries.Geometry ToNTSGeometry();

}

// 2. DATABASE MODELS (Entity Framework)

public class ParcelGeometry

{

public int GeometryId { get; set; }

public int ParcelId { get; set; }

// NTS geometry column (stored as WKB in SQLite)

```

[Column(TypeName = "geometry")]
public NetTopologySuite.Geometries.Polygon Geometry { get; set; }

// Computed properties
public double AreaSqm => Geometry.Area;
public double Perimeter => Geometry.Length;
}

// 3. SPATIAL INDEX (Already implemented - RTreeSpatialIndex.cs)
public class RTreeSpatialIndex
{
    private STRtree<IShape> _index; // ← NTS R-Tree!

    public List<IShape> Query(RectangleD area)
    {
        var env = new Envelope(area.X, area.X + area.Width,
                                area.Y, area.Y + area.Height);
        return _index.Query(env).ToList();
    }
}

```

4. COMPLETE NTS CLASS REFERENCE

4.1 Core Geometry Classes

Point - Single coordinate location

using NetTopologySuite.Geometries;

// What it represents:

// - Parcel corner

// - Survey benchmark

// - Utility pole location

// Creating points:

var factory = new GeometryFactory();

Point corner = factory.CreatePoint(new Coordinate(535123.45, 3012456.78));

// Properties:

double x = corner.X; // 535123.45

double y = corner.Y; // 3012456.78

Coordinate coord = corner.Coordinate;

int dimension = corner.Dimension; // 0 (point is 0D)

// Methods:

Point buffered = (Point)corner.Buffer(10.0); // Creates circle of radius 10m

double distance = corner.Distance(otherPoint);

bool within = corner.Within(polygon);

When to use:

- Marking survey points

- Storing parcel centroids
- GPS waypoints
- Property boundary markers

LineString - Connected sequence of points

// What it represents:

// - Parcel boundary (open)

// - Road centerline

// - Utility line

// - Fence line

// Creating linestrings:

```
var coords = new Coordinate[]
```

```
{
```

```
    new Coordinate(0, 0),
```

```
    new Coordinate(100, 0),
```

```
    new Coordinate(100, 50),
```

```
    new Coordinate(0, 50)
```

```
};
```

```
LineString boundary = factory.CreateLineString(coords);
```

// Properties:

```
int numPoints = boundary.NumPoints;    // 4
```

```
double length = boundary.Length;    // 300 (perimeter if closed)
```

```
Point startPt = boundary.StartPoint;    // First coordinate
```

```
Point endPt = boundary.EndPoint;    // Last coordinate
```

```
bool isClosed = boundary.IsClosed;    // false (start ≠ end)
```

```
bool isRing = boundary.IsRing;    // false (closed + simple)
```

```
Coordinate[] vertices = boundary.Coordinates;
```

// Methods:

```
LineString simplified = (LineString)boundary.Simplify(0.5);
```

```
Point centroid = boundary.Centroid;
```

```
Geometry buffer = boundary.Buffer(5.0);    // Creates polygon around line
```

When to use:

- Roads, paths, boundaries (before closing to polygon)
- Any linear feature
- When you need endpoints distinct

LinearRing - Closed LineString (MUST be simple)

// What it represents:

// - Parcel boundary (MUST start/end at same point)

// - Building footprint outline

// - Block boundary

// Creating rings (CRITICAL: Last coord = First coord):

```
var ringCoords = new Coordinate[]
```

```

{
    new Coordinate(0, 0),
    new Coordinate(100, 0),
    new Coordinate(100, 50),
    new Coordinate(0, 50),
    new Coordinate(0, 0) // ← MUST CLOSE!
};
LinearRing ring = factory.CreateLinearRing(ringCoords);

// Properties:
bool isValid = ring.IsValid;    // true if no self-intersections
bool isClosed = ring.IsClosed; // always true by definition
bool isSimple = ring.IsSimple; // true if no self-crossings

// Validation:
var validator = new IsValidOp(ring);
if (!validator.IsValid)
{
    var error = validator.ValidationError;
    // error.ErrorType: "Self-intersection", "Ring not closed", etc.
    // error.Coordinate: Where the problem is
}

```

When to use:

- ALWAYS for creating polygon shells
- Ensures topologically valid parcels

Polygon - Area with holes (the MOST IMPORTANT for parcels)

```

// What it represents:
// - Land parcel
// - Building footprint
// - Zoning area
// - Block

// Creating simple polygon (no holes):
LinearRing shell = factory.CreateLinearRing(coords);
Polygon parcel = factory.CreatePolygon(shell);

// Creating polygon WITH HOLES (e.g., parcel with building):
LinearRing outerBoundary = factory.CreateLinearRing(parcelCoords);
LinearRing[] holes = new LinearRing[]
{
    factory.CreateLinearRing(buildingCoords),
    factory.CreateLinearRing(pondCoords)
};
Polygon parcelWithHoles = factory.CreatePolygon(outerBoundary, holes);

// Properties:

```

```

double area = parcel.Area;           // Square meters
double perimeter = parcel.Length;    // Meters
Point centroid = parcel.Centroid;    // Geometric center
Coordinate[] coords = parcel.Coordinates; // All vertices
LinearRing exteriorRing = parcel.ExteriorRing;
int numHoles = parcel.NumInteriorRings;
LinearRing hole = parcel.GetInteriorRingN(0);

```

// Critical Methods:

```

bool containsPt = parcel.Contains(point);
bool intersects = parcel.Intersects(otherParcel);
bool touches = parcel.Touches(adjacentParcel);
bool overlaps = parcel.Overlaps(otherParcel);
Geometry intersection = parcel.Intersection(otherParcel);
Geometry union = parcel.Union(otherParcel);
Geometry difference = parcel.Difference(roadArea);
Geometry buffer = parcel.Buffer(5.0); // Setback/offset
Geometry convexHull = parcel.ConvexHull;

```

When to use:

- 95% of your parcel operations
- Any area calculation
- Merging/splitting parcels

MultiPoint, MultiLineString, MultiPolygon - Collections

// What they represent:

// - MultiPoint: Survey markers for one project

// - MultiLineString: Road network

// - MultiPolygon: Non-contiguous parcels owned by same person

// Creating multi-geometries:

```

Point[] surveyPoints = new Point[] { pt1, pt2, pt3 };
MultiPoint markers = factory.CreateMultiPoint(surveyPoints);

```

```

Polygon[] parcels = new Polygon[] { parcel1, parcel2, parcel3 };
MultiPolygon estate = factory.CreateMultiPolygon(parcels);

```

// Properties:

```

int count = estate.NumGeometries;
Polygon first = (Polygon)estate.GetGeometryN(0);
double totalArea = estate.Area; // Sum of all polygons

```

// Methods (same as single geometries):

```

Geometry merged = estate.Union(); // Merge all into one

```

When to use:

- Owner has multiple non-adjacent parcels
 - Managing collections of related geometries
-

GeometryCollection - Mixed geometry types

// What it represents:

// - Survey data: points + lines + polygons together

// - Import data with mixed types

```
Geometry[] mixed = new Geometry[] { point1, line1, polygon1 };
```

```
GeometryCollection collection = factory.CreateGeometryCollection(mixed);
```

// Iterate through mixed types:

```
for (int i = 0; i < collection.NumGeometries; i++)
```

```
{  
    Geometry geom = collection.GetGeometryN(i);
```

```
    if (geom is Point pt)
```

```
        ProcessPoint(pt);
```

```
    else if (geom is LineString ln)
```

```
        ProcessLine(ln);
```

```
    else if (geom is Polygon pg)
```

```
        ProcessParcel(pg);
```

```
}
```

When to use:

- Importing CAD drawings (mixed geometries)
- Temporary storage during complex operations

4.2 Essential Support Classes

Coordinate - Basic building block

// Represents a single (X, Y) or (X, Y, Z) point

// Creation:

```
Coordinate c1 = new Coordinate(535123.45, 3012456.78);
```

```
Coordinate c2 = new Coordinate(535123.45, 3012456.78, 125.5); // With elevation
```

// Properties:

```
double x = c1.X;
```

```
double y = c1.Y;
```

```
double z = c1.Z; // NaN if not set
```

// Methods:

```
double dist = c1.Distance(c2);
```

```
bool equals = c1.Equals2D(c2); // Ignores Z
```

```
bool equals3D = c1.Equals3D(c2); // Includes Z
```

Envelope - Bounding box (axis-aligned rectangle)

// Represents min/max extents of geometry

// Creation:

```
Envelope bbox = new Envelope(minX, maxX, minY, maxY);
```



```
Envelope fromGeom = parcel.EnvelopeInternal;
```

```
// Properties:
```

```
double width = bbox.Width;  
double height = bbox.Height;  
double area = bbox.Area;  
Coordinate center = bbox.Centre;  
double minX = bbox.MinX;  
double maxX = bbox.MaxX;  
double minY = bbox.MinY;  
double maxY = bbox.MaxY;
```

```
// Methods:
```

```
bool intersects = bbox.Intersects(otherBbox);  
bool contains = bbox.Contains(point.Coordinate);  
bbox.ExpandToInclude(newCoordinate);  
Envelope intersection = bbox.Intersection(otherBbox);
```

Critical for performance:

- Quick "is parcel in viewport?" checks
- Spatial index (R-Tree) queries
- Much faster than full geometry intersection

GeometryFactory - Object creation

```
// Factory pattern for creating all geometries
```

```
// Creation (precision model):
```

```
var factory = new GeometryFactory();           // Default precision  
var preciseFactory = new GeometryFactory(  
    new PrecisionModel(PrecisionModels.Floating), // Full double precision  
    4326                                           // SRID (coordinate system)  
);
```

```
// Methods (already shown above):
```

```
Point pt = factory.CreatePoint(coord);  
LineString line = factory.CreateLineString(coords);  
LinearRing ring = factory.CreateLinearRing(coords);  
Polygon poly = factory.CreatePolygon(shell, holes);
```

Best Practice:

- Create ONE factory and reuse it
- Set SRID early (coordinate reference system ID)

PrecisionModel - Coordinate precision

```
// Controls rounding behavior
```

```
// Types:
```

```
PrecisionModel floating = new PrecisionModel(PrecisionModels.Floating);  
// ↑ Full double precision (15-17 significant digits)
```

```
PrecisionModel fixed = new PrecisionModel(PrecisionModels.Fixed);  
// ↑ Rounds to nearest integer coordinate
```

```
PrecisionModel scale = new PrecisionModel(1000.0);  
// ↑ Rounds to 3 decimal places (mm accuracy)
```

```
// Usage:
```

```
var factory = new GeometryFactory(precisionModel, srid);
```

Recommendation for land surveying: Use **Floating** with 1mm scale for Nepal TM projection:

```
var precision = new PrecisionModel(1000.0); // 1000 = mm precision
```

```
var factory = new GeometryFactory(precision, 6207); // Nepal TM SRID
```

4.3 Spatial Operations

Topological Predicates - Relationship testing

```
// All return bool (true/false)
```

```
// Spatial relationships:
```

```
bool contains = parcel1.Contains(parcel2);
```

```
// ↑ parcel2 is completely inside parcel1 (no boundary touch)
```

```
bool within = parcel2.Within(parcel1);
```

```
// ↑ Inverse of Contains
```

```
bool intersects = parcel1.Intersects(parcel2);
```

```
// ↑ Geometries share any point (interior or boundary)
```

```
bool overlaps = parcel1.Overlaps(parcel2);
```

```
// ↑ Share area but neither contains the other (ambiguous ownership!)
```

```
bool touches = parcel1.Touches(parcel2);
```

```
// ↑ Share boundary but not interior (adjacent parcels)
```

```
bool crosses = road.Crosses(parcel);
```

```
// ↑ Line crosses polygon boundary
```

```
bool disjoint = parcel1.Disjoint(parcel2);
```

```
// ↑ No shared points at all (completely separate)
```

```
bool equals = parcel1.Equals(parcel2);
```

```
// ↑ Same geometry (topologically equivalent)
```

```
bool covers = parcel1.Covers(point);
```

```
// ↑ Like Contains but boundary points count
```

```
// Distance:
```

```
double distance = parcel1.Distance(parcel2);
```

```
// ↑ Shortest distance between geometries (0 if touching)

bool isWithinDist = parcel1.IsWithinDistance(parcel2, 10.0);
// ↑ Optimized "distance < threshold" check
Real-world examples:
// Find all parcels touching a selected parcel:
var adjacent = allParcels.Where(p => p.Touches(selectedParcel));

// Find parcels within 50m of a road:
var nearRoad = parcels.Where(p => p.Distance(road) <= 50.0);

// Detect ownership disputes (overlapping parcels):
var disputes = parcels.Where(p => p.Overlaps(selectedParcel));
```

Geometric Operations - Shape manipulation

```
// All return new Geometry objects (immutable operations)

// Buffer (offset):
Geometry setback = parcel.Buffer(5.0);
// ↑ Creates new polygon 5m OUTSIDE boundary (positive)

Geometry shrunk = parcel.Buffer(-2.0);
// ↑ Creates polygon 2m INSIDE boundary (negative)

// Union (merge):
Geometry combined = parcel1.Union(parcel2);
// ↑ Merges two parcels into one (removes shared boundary)

Geometry allMerged = factory.CreateGeometryCollection(parcel1, parcel2).Union();
// ↑ Merges multiple parcels

// Intersection (overlap area):
Geometry overlap = parcel1.Intersection(parcel2);
// ↑ Returns ONLY the area shared by both

// Difference (subtraction):
Geometry remainder = parcel.Difference(roadArea);
// ↑ Cuts out road area from parcel

// Symmetric Difference (XOR):
Geometry symDiff = parcel1.SymmetricDifference(parcel2);
// ↑ Everything NOT in intersection (opposite of intersection)

// Convex Hull:
Geometry hull = parcel.ConvexHull();
// ↑ Smallest convex polygon containing all vertices
```

```

// Boundary:
Geometry boundary = parcel.Boundary;
// ↑ Returns LineString of exterior ring (for area calculations)

// Centroid:
Point center = parcel.Centroid;
// ↑ Geometric center (may be outside for concave shapes!)

// Interior Point:
Point inside = parcel.InteriorPoint;
// ↑ GUARANTEED to be inside polygon (for labeling)
Real-world examples:
// Merge all parcels owned by same person:
var ownerParcels = GetParcelsByOwner(ownerId);
Geometry estate = factory.CreateGeometryCollection(ownerParcels).Union();

// Create road reservation (20m from centerline):
Geometry roadArea = centerline.Buffer(10.0); // 10m each side = 20m total

// Calculate net buildable area (parcel minus setbacks):
Polygon buildable = (Polygon)parcel.Buffer(-5.0); // 5m setback
double netArea = buildable.Area;

// Find area lost to road widening:
Geometry oldRoad = oldCenterline.Buffer(8.0); // 16m wide
Geometry newRoad = newCenterline.Buffer(12.0); // 24m wide
Geometry lostArea = newRoad.Difference(oldRoad);

```

Validation - Topology checking

```

// IsValid property (quick check):
bool valid = parcel.IsValid;

// Detailed validation:
var validator = new IsValidOp(parcel);
if (!validator.IsValid)
{
    TopologyValidationError error = validator.ValidationError;

    // Error types:
    // - "Ring Self-intersection"
    // - "Hole lies outside shell"
    // - "Interior is disconnected"
    // - "Self-intersection"
    // - "Ring not closed"
    // - "Duplicate Points"

    string errorType = error.ErrorType;

```

```

Coordinate location = error.Coordinate; // Where error occurs

Console.WriteLine($"Invalid: {errorType} at ({location.X}, {location.Y})");
}

// Make invalid geometry valid:
Geometry fixed = parcel.Buffer(0); // Magic trick!
// ↑ Buffer(0) forces topology correction

// Or use explicit fixer:
var fixer = new GeometryFixer(parcel);
Geometry repaired = fixer.GetResult();
Common validity errors in land data:
// 1. Self-intersecting polygon (bowtie shape):
//   Fix: Split into multiple parcels or buffer(0)

// 2. Duplicate consecutive points:
//   Fix: Remove duplicates before creating geometry

// 3. Unclosed ring:
//   Fix: Add closing coordinate (first = last)

// 4. Hole outside shell:
//   Fix: Rebuild polygon with correct holes

```

4.4 I/O Classes - Reading/Writing

WKT (Well-Known Text) - Human-readable format

using NetTopologySuite.IO;

```

// Writing geometry to WKT:
var writer = new WKTWriter();
string wkt = writer.Write(parcel);
// Output: "POLYGON ((0 0, 100 0, 100 50, 0 50, 0 0))"

// Reading WKT:
var reader = new WKTReader();
Geometry geom = reader.Read("POLYGON ((0 0, 100 0, 100 50, 0 50, 0 0))");
When to use:

- Debugging (print geometry to console)
- Text file import/export
- SQL queries (human-readable)

```

WKB (Well-Known Binary) - Compact database storage

```

// Writing to WKB (for database):
var writer = new WKBWriter();
byte[] wkb = writer.Write(parcel);

```

```
// Reading from WKB:
var reader = new WKBReader();
Geometry geom = reader.Read(wkb);
```

When to use:

- **Database storage (ALWAYS)**
- Network transmission
- File storage (binary files)

Why WKB over WKT?

- 50-70% smaller file size
 - Faster parsing
 - No precision loss from text conversion
-

GeoJSON - Web/JavaScript interop

using NetTopologySuite.IO;

```
// Writing:
var writer = new GeoJsonWriter();
string json = writer.Write(parcel);
// Output: {"type":"Polygon","coordinates": [[[0,0],[100,0]...]]}
```

```
// Reading:
var reader = new GeoJsonReader();
Geometry geom = reader.Read<Geometry>(json);
```

When to use:

- Web mapping (Leaflet, Mapbox, OpenLayers)
 - REST APIs
 - JavaScript visualization
-

4.5 Spatial Indexing

STRtree - R-Tree spatial index (CRITICAL for performance)

using NetTopologySuite.Index.Strtree;

```
// Create index:
var index = new STRtree<Parcel>();

// Insert all parcels:
foreach (var parcel in allParcels)
{
    Envelope bbox = parcel.Geometry.EnvelopeInternal;
    index.Insert(bbox, parcel);
}
```

```
// Build index (MUST call after all inserts):
index.Build(); // ← Makes queries fast!
```

```
// Query by bounding box (viewport culling):
Envelope viewport = new Envelope(minX, maxX, minY, maxY);
```

```

var visibleParcels = index.Query(viewport);

// Query by geometry:
var nearbyParcels = index.Query(selectedParcel.Geometry.EnvelopeInternal);

// Precise filtering (R-Tree returns candidates, not exact):
var exactMatches = nearbyParcels
    .Where(p => p.Geometry.Intersects(selectedParcel.Geometry))
    .ToList();
Performance:

- Without index:  $O(n)$  - Check ALL parcels
- With R-Tree:  $O(\log n)$  - Check only candidates

When you have 10,000 parcels:

- No index: 10,000 checks per selection
- R-Tree: ~50-100 checks per selection

Example: Find all parcels within 100m of a point:
Point location = factory.CreatePoint(new Coordinate(x, y));
Envelope searchArea = new Envelope(
    location.X - 100, location.X + 100,
    location.Y - 100, location.Y + 100
);

var candidates = index.Query(searchArea);
var parcelsWithin100m = candidates
    .Where(p => p.Geometry.Distance(location) <= 100.0)
    .ToList();

```

5. DATABASE INTEGRATION WITH ENTITY FRAMEWORK

5.1 NuGet Packages Required

```

<!-- In your .csproj file -->
<ItemGroup>
    <PackageReference Include="NetTopologySuite" Version="2.5.0" />
    <PackageReference Include="NetTopologySuite.IO.GeoJSON" Version="4.0.0" />
    <PackageReference Include="NetTopologySuite.IO.SqlServerBytes" Version="2.1.0" />

    <!-- Entity Framework Core -->
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="7.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="7.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="7.0.0" />

    <!-- Spatial support for EF Core -->
    <PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite.NetTopologySuite"
Version="7.0.0" />
</ItemGroup>

```

5.2 Entity Classes

```

using NetTopologySuite.Geometries;
using System.ComponentModel.DataAnnotations;

```

```
using System.ComponentModel.DataAnnotations.Schema;
```

```
namespace RePlot.Data.Entities
```

```
{
```

```
    /// <summary>
```

```
    /// Parcel geometry stored in database
```

```
    /// </summary>
```

```
    public class ParcelGeometry
```

```
    {
```

```
        [Key]
```

```
        public int GeometryId { get; set; }
```

```
        [Required]
```

```
        public int ParcelId { get; set; }
```

```
        [Required]
```

```
        public int LayerId { get; set; }
```

```
        /// <summary>
```

```
        /// NTS Polygon stored as WKB in SQLite
```

```
        /// </summary>
```

```
        [Column(TypeName = "geometry")]
```

```
        [Required]
```

```
        public Polygon Geometry { get; set; }
```

```
        /// <summary>
```

```
        /// Computed property - no database column
```

```
        /// </summary>
```

```
        [NotMapped]
```

```
        public double AreaSqm => Geometry?.Area ?? 0;
```

```
        [NotMapped]
```

```
        public double Perimeter => Geometry?.Length ?? 0;
```

```
        [NotMapped]
```

```
        public Point Centroid => Geometry?.Centroid;
```

```
        // Bounding box (for R-Tree indexing)
```

```
        public double BBoxMinX { get; set; }
```

```
        public double BBoxMinY { get; set; }
```

```
        public double BBoxMaxX { get; set; }
```

```
        public double BBoxMaxY { get; set; }
```

```
        // Navigation properties
```

```
        [ForeignKey(nameof(ParcelId))]
```

```
        public OriginalLandParcel Parcel { get; set; }
```



```

[ForeignKey(nameof(LayerId))]
public Layer Layer { get; set; }

/// <summary>
/// Update bounding box from geometry
/// </summary>
public void UpdateBoundingBox()
{
    if (Geometry != null)
    {
        var env = Geometry.EnvelopeInternal;
        BBoxMinX = env.MinX;
        BBoxMinY = env.MinY;
        BBoxMaxX = env.MaxX;
        BBoxMaxY = env.MaxY;
    }
}

}

/// <summary>
/// Road centerline geometry
/// </summary>
public class RoadCenterline
{
    [Key]
    public int RoadId { get; set; }

    public string RoadName { get; set; }

    [Required]
    public double Width { get; set; }

    public double CornerRadius { get; set; }

    /// <summary>
    /// NTS LineString for road centerline
    /// </summary>
    [Column(TypeName = "geometry")]
    [Required]
    public LineString Geometry { get; set; }

    public double BBoxMinX { get; set; }
    public double BBoxMinY { get; set; }
    public double BBoxMaxX { get; set; }
    public double BBoxMaxY { get; set; }

    public DateTime CreatedDate { get; set; }

```

```

[NotMapped]
public double Length => Geometry?.Length ?? 0;

[NotMapped]
public Polygon RoadSurface => (Polygon)Geometry?.Buffer(Width / 2.0);
}

/// <summary>
/// Block boundary geometry
/// </summary>
public class Block
{
    [Key]
    public int BlockId { get; set; }

    public string BlockNumber { get; set; }

    [Column(TypeName = "geometry")]
    [Required]
    public Polygon Geometry { get; set; }

    public double BBoxMinX { get; set; }
    public double BBoxMinY { get; set; }
    public double BBoxMaxX { get; set; }
    public double BBoxMaxY { get; set; }

    public DateTime CreatedDate { get; set; }

    [NotMapped]
    public double AreaSqm => Geometry?.Area ?? 0;
}
}

```

5.3 DbContext Configuration

```

using Microsoft.EntityFrameworkCore;
using NetTopologySuite;
using NetTopologySuite.Geometries;

namespace RePlot.Data
{
    public class ReplotDbContext : DbContext
    {
        public DbSet<ParcelGeometry> ParcelGeometries { get; set; }
        public DbSet<RoadCenterline> RoadCenterlines { get; set; }
        public DbSet<Block> Blocks { get; set; }
        public DbSet<OriginalLandParcel> OriginalParcels { get; set; }
        public DbSet<Layer> Layers { get; set; }
    }
}

```

```

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    // Configure SQLite with spatial support
    optionsBuilder.UseSqlite(
        "Data Source=replot.db",
        x => x.UseNetTopologySuite() // ← CRITICAL: Enable NTS support
    );
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    // Configure geometry columns
    modelBuilder.Entity<ParcelGeometry>(entity =>
    {
        entity.ToTable("tblParcelGeometry");

        // Spatial index on Geometry column
        entity.HasIndex(e => e.Geometry)
            .HasDatabaseGeneratedOption(DatabaseGeneratedOption.None);

        // Regular indexes on bounding box for fast filtering
        entity.HasIndex(e => new { e.BBoxMinX, e.BBoxMinY, e.BBoxMaxX, e.BBoxMaxY });
    });

    modelBuilder.Entity<RoadCenterline>(entity =>
    {
        entity.ToTable("tblRoadCenterlines");
        entity.HasIndex(e => e.Geometry);
    });

    modelBuilder.Entity<Block>(entity =>
    {
        entity.ToTable("tblBlocks");
        entity.HasIndex(e => e.Geometry);
    });
}
}

```

5.4 Migrations

```

# Add migration with spatial support
dotnet ef migrations add InitialCreate_WithSpatial

```

```

# Update database
dotnet ef database update

```

Generated SQL (SQLite):

```
CREATE TABLE "tblParcelGeometry" (  
    "GeometryId" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,  
    "ParcelId" INTEGER NOT NULL,  
    "LayerId" INTEGER NOT NULL,  
    "Geometry" BLOB NOT NULL, -- WKB binary data  
    "BBoxMinX" REAL NOT NULL,  
    "BBoxMinY" REAL NOT NULL,  
    "BBoxMaxX" REAL NOT NULL,  
    "BBoxMaxY" REAL NOT NULL,  
    FOREIGN KEY("ParcelId") REFERENCES "tblOriginalLandParcels"("ParcelId") ON DELETE  
    CASCADE  
);
```

```
CREATE INDEX "IX_tblParcelGeometry_BBox"  
ON "tblParcelGeometry" ("BBoxMinX", "BBoxMinY", "BBoxMaxX", "BBoxMaxY");
```

5.5 CRUD Operations

```
using (var context = new ReplotDbContext())  
{  
    var factory = NtsGeometryServices.Instance.CreateGeometryFactory(srid: 6207);  
  
    // CREATE  
    var coords = new Coordinate[]  
    {  
        new Coordinate(535100, 3012400),  
        new Coordinate(535150, 3012400),  
        new Coordinate(535150, 3012450),  
        new Coordinate(535100, 3012450),  
        new Coordinate(535100, 3012400) // Close the ring  
    };  
  
    var ring = factory.CreateLinearRing(coords);  
    var polygon = factory.CreatePolygon(ring);  
  
    var parcelGeom = new ParcelGeometry  
    {  
        ParcelId = 123,  
        LayerId = 1,  
        Geometry = polygon  
    };  
    parcelGeom.UpdateBoundingBox();  
  
    context.ParcelGeometries.Add(parcelGeom);  
    context.SaveChanges();  
  
    // READ  
    var geom = context.ParcelGeometries
```

```

        .Include(p => p.Parcel)
        .FirstOrDefault(p => p.GeometryId == 1);

Console.WriteLine($"Area: {geom.AreaSqm} sq.m");

// UPDATE
geom.Geometry = geom.Geometry.Buffer(-2.0) as Polygon; // 2m setback
geom.UpdateBoundingBox();
context.SaveChanges();

// DELETE
context.ParcelGeometries.Remove(geom);
context.SaveChanges();
}
5.6 Spatial Queries
// Query parcels intersecting a viewport
Envelope viewport = new Envelope(minX, maxX, minY, maxY);
var viewportPoly = factory.CreatePolygon(
    factory.CreateLinearRing(new[]
    {
        new Coordinate(viewport.MinX, viewport.MinY),
        new Coordinate(viewport.MaxX, viewport.MinY),
        new Coordinate(viewport.MaxX, viewport.MaxY),
        new Coordinate(viewport.MinX, viewport.MaxY),
        new Coordinate(viewport.MinX, viewport.MinY)
    })
);

var visibleParcels = context.ParcelGeometries
    .Where(p => p.Geometry.Intersects(viewportPoly))
    .ToList();

// Find parcels touching a selected parcel
var adjacentParcels = context.ParcelGeometries
    .Where(p => p.GeometryId != selectedId && p.Geometry.Touches(selectedGeometry))
    .ToList();

// Find parcels within distance
double bufferDistance = 50.0; // 50 meters
var bufferGeom = selectedGeometry.Buffer(bufferDistance);

var nearbyParcels = context.ParcelGeometries
    .Where(p => p.Geometry.Intersects(bufferGeom))
    .ToList();

// Calculate total area by layer
var layerAreas = context.ParcelGeometries

```

```

.GroupBy(p => p.LayerId)
.Select(g => new
{
    LayerId = g.Key,
    TotalArea = g.Sum(p => p.Geometry.Area)
})
.ToList();

```

6. PARCEL GEOMETRY OPERATIONS

6.1 Creating Parcels

```

public class ParcelCreationService
{
    private readonly GeometryFactory _factory;

    public ParcelCreationService()
    {
        var precision = new PrecisionModel(1000.0); // 1mm precision
        _factory = new GeometryFactory(precision, 6207); // Nepal TM
    }

    /// <summary>
    /// Create parcel from coordinate array
    /// </summary>
    public Polygon CreateParcel(Coordinate[] coordinates)
    {
        // Validation
        if (coordinates == null || coordinates.Length < 4)
            throw new ArgumentException("Parcel needs at least 4 coordinates");

        // Ensure closed ring
        if (!coordinates[0].Equals2D(coordinates[coordinates.Length - 1]))
        {
            var closedCoords = new Coordinate[coordinates.Length + 1];
            Array.Copy(coordinates, closedCoords, coordinates.Length);
            closedCoords[coordinates.Length] = new Coordinate(coordinates[0]);
            coordinates = closedCoords;
        }

        // Create geometry
        var ring = _factory.CreateLinearRing(coordinates);
        var polygon = _factory.CreatePolygon(ring);

        // Validate
        if (!polygon.IsValid)
        {
            var validator = new IsValidOp(polygon);
            throw new InvalidOperationException(

```

```

        $"Invalid geometry: {validator.ValidationError.ErrorType} " +
        $"at ({validator.ValidationError.Coordinate.X},
{validator.ValidationError.Coordinate.Y})"
    );
}

return polygon;
}

/// <summary>
/// Create parcel from IShape (your existing drawing objects)
/// </summary>
public Polygon CreateParcelFromShape(PolylineShape polyline)
{
    var coords = polyline.Vertices
        .Select(v => new Coordinate(v.X, v.Y))
        .ToArray();

    return CreateParcel(coords);
}

/// <summary>
/// Create rectangle parcel
/// </summary>
public Polygon CreateRectangle(double minX, double minY, double width, double height)
{
    var coords = new[]
    {
        new Coordinate(minX, minY),
        new Coordinate(minX + width, minY),
        new Coordinate(minX + width, minY + height),
        new Coordinate(minX, minY + height),
        new Coordinate(minX, minY) // Close
    };

    return CreateParcel(coords);
}
}

```

6.2 Merging Parcels

```

public class ParcelMergeService
{
    /// <summary>
    /// Merge multiple parcels into one (removes internal boundaries)
    /// </summary>
    public Polygon MergeParcels(IEnumerable<Polygon> parcels)
    {
        var factory = parcels.First().Factory;
    }
}

```

```

var collection = factory.CreateGeometryCollection(parcel.ToArray());

// Union dissolves shared boundaries
Geometry merged = collection.Union();

// May return MultiPolygon if parcels don't touch
if (merged is Polygon polygon)
    return polygon;
else if (merged is MultiPolygon multi && multi.NumGeometries == 1)
    return (Polygon)multi.GetGeometryN(0);
else
    throw new InvalidOperationException(
        "Parcels are not adjacent - result is multiple polygons");
}

/// <summary>
/// Merge only if parcels touch
/// </summary>
public Polygon? TryMergeParcels(Polygon parcel1, Polygon parcel2)
{
    // Check adjacency
    if (!parcel1.Touches(parcel2) && !parcel1.Overlaps(parcel2))
        return null;

    var merged = parcel1.Union(parcel2);

    if (merged is Polygon polygon)
        return polygon;
    else
        return null;
}
}

```

Real-world example:

```

// Owner buys adjacent parcel - merge them
var ownerParcels = GetParcelsByOwner(ownerId);
var mergedEstate = mergeService.MergeParcels(ownerParcels);

// Update database
using (var context = new ReplotDbContext())
{
    // Delete old parcels
    foreach (var parcel in ownerParcels)
        context.ParcelGeometries.Remove(parcel);

    // Insert merged parcel
    var newParcel = new ParcelGeometry
    {

```



```

        ParcelId = newParcelId,
        LayerId = layerId,
        Geometry = mergedEstate
    };
    newParcel.UpdateBoundingBox();
    context.ParcelGeometries.Add(newParcel);

    context.SaveChanges();
}

```

6.3 Splitting Parcels

```

public class ParcelSplitService
{
    /// <summary>
    /// Split parcel along a line
    /// </summary>
    public List<Polygon> SplitParcel(Polygon parcel, LineString splitLine)
    {
        // Buffer the line to create a "blade"
        var blade = splitLine.Buffer(0.001); // 1mm wide blade

        // Split creates multiple pieces
        var difference = parcel.Difference(blade);

        var result = new List<Polygon>();

        if (difference is Polygon singlePoly)
        {
            result.Add(singlePoly);
        }
        else if (difference is MultiPolygon multi)
        {
            for (int i = 0; i < multi.NumGeometries; i++)
                result.Add((Polygon)multi.GetGeometryN(i));
        }
        else if (difference is GeometryCollection collection)
        {
            for (int i = 0; i < collection.NumGeometries; i++)
            {
                if (collection.GetGeometryN(i) is Polygon poly)
                    result.Add(poly);
            }
        }

        return result;
    }

    /// <summary>

```

```

/// Split parcel into N equal-area pieces (subdivision)
/// </summary>
public List<Polygon> SubdivideParcel(Polygon parcel, int numPieces)
{
    // This is complex - simplified version:
    // 1. Get parcel envelope
    // 2. Create grid
    // 3. Intersect with parcel
    // 4. Adjust to equalize areas

    var env = parcel.EnvelopeInternal;
    double width = env.Width / Math.Sqrt(numPieces);
    double height = env.Height / Math.Sqrt(numPieces);

    var pieces = new List<Polygon>();
    var factory = parcel.Factory;

    for (double x = env.MinX; x < env.MaxX; x += width)
    {
        for (double y = env.MinY; y < env.MaxY; y += height)
        {
            var cell = factory.CreatePolygon(
                factory.CreateLinearRing(new[]
                {
                    new Coordinate(x, y),
                    new Coordinate(x + width, y),
                    new Coordinate(x + width, y + height),
                    new Coordinate(x, y + height),
                    new Coordinate(x, y)
                })
            );

            var intersection = parcel.Intersection(cell);
            if (intersection is Polygon piece && piece.Area > 0.01)
                pieces.Add(piece);
        }
    }

    return pieces;
}

```

6.4 Adjusting Parcel Boundaries

```

public class BoundaryAdjustmentService
{
    /// <summary>
    /// Apply setback to create building envelope
    /// </summary>

```

```

public Polygon ApplySetback(Polygon parcel, double setbackDistance)
{
    // Negative buffer shrinks polygon
    var setback = parcel.Buffer(-setbackDistance);

    if (setback is Polygon polygon)
        return polygon;
    else if (setback.IsEmpty)
        throw new InvalidOperationException("Setback too large - no area remaining");
    else
        throw new InvalidOperationException("Setback created multiple polygons");
}

/// <summary>
/// Expand parcel boundary (land acquisition)
/// </summary>
public Polygon ExpandBoundary(Polygon parcel, double expansionDistance)
{
    var expanded = parcel.Buffer(expansionDistance);

    if (expanded is Polygon polygon)
        return polygon;
    else
        throw new InvalidOperationException("Expansion created invalid geometry");
}

/// <summary>
/// Simplify complex boundary (reduce vertices)
/// </summary>
public Polygon SimplifyBoundary(Polygon parcel, double tolerance)
{
    // Douglas-Peucker algorithm
    var simplified = DouglasPeuckerSimplifier.Simplify(parcel, tolerance);

    if (simplified is Polygon polygon)
        return polygon;
    else
        throw new InvalidOperationException("Simplification failed");
}

/// <summary>
/// Square up parcel corners (regularize)
/// </summary>
public Polygon RegularizeCorners(Polygon parcel, double angleTolerance = 5.0)
{
    // Get exterior coordinates
    var coords = parcel.ExteriorRing.Coordinates;

```

```

var adjusted = new List<Coordinate>();

for (int i = 0; i < coords.Length - 1; i++)
{
    var prev = i > 0 ? coords[i - 1] : coords[coords.Length - 2];
    var curr = coords[i];
    var next = coords[i + 1];

    // Calculate angles
    double angle = CalculateAngle(prev, curr, next);

    // If close to 90°, snap to 90°
    if (Math.Abs(angle - 90) < angleTolerance)
    {
        // Adjust coordinate to make 90° angle
        var adjustedCoord = SnapTo90Degrees(prev, curr, next);
        adjusted.Add(adjustedCoord);
    }
    else
    {
        adjusted.Add(curr);
    }
}

adjusted.Add(adjusted[0]); // Close ring

var ring = parcel.Factory.CreateLinearRing(adjusted.ToArray());
return parcel.Factory.CreatePolygon(ring);
}

private double CalculateAngle(Coordinate p1, Coordinate vertex, Coordinate p2)
{
    double angle1 = Math.Atan2(p1.Y - vertex.Y, p1.X - vertex.X);
    double angle2 = Math.Atan2(p2.Y - vertex.Y, p2.X - vertex.X);
    double angle = Math.Abs(angle2 - angle1) * 180 / Math.PI;
    return angle > 180 ? 360 - angle : angle;
}

private Coordinate SnapTo90Degrees(Coordinate prev, Coordinate curr, Coordinate next)
{
    // Simplified - actual implementation needs robust math
    return curr; // Placeholder
}
}

6.5 Road Network Operations
public class RoadNetworkService
{

```

```

/// <summary>
/// Create road surface from centerline
/// </summary>
public Polygon CreateRoadSurface(LineString centerline, double roadWidth)
{
    return (Polygon)centerline.Buffer(roadWidth / 2.0);
}

/// <summary>
/// Create road with rounded corners at intersections
/// </summary>
public Polygon CreateRoadWithFilletts(LineString centerline, double roadWidth, double
cornerRadius)
{
    // Buffer with end cap and join styles
    var bufferParams = new BufferParameters
    {
        EndCapStyle = EndCapStyle.Flat,
        JoinStyle = JoinStyle.Round,
        QuadrantSegments = 16 // Smoothness of corners
    };

    var buffer = BufferOp.Buffer(centerline, roadWidth / 2.0, bufferParams);
    return (Polygon)buffer;
}

/// <summary>
/// Subtract road area from parcel
/// </summary>
public Polygon SubtractRoadFromParcel(Polygon parcel, Polygon roadSurface)
{
    var remainder = parcel.Difference(roadSurface);

    if (remainder is Polygon polygon)
        return polygon;
    else if (remainder is MultiPolygon multi)
    {
        // Road split parcel - return largest piece
        var largest = multi.Geometries
            .Cast<Polygon>()
            .OrderByDescending(p => p.Area)
            .First();
        return largest;
    }
    else
        throw new InvalidOperationException("Road subtraction failed");
}

```

```

/// <summary>
/// Calculate road area taken from each parcel
/// </summary>
public Dictionary<int, double> CalculateRoadContributions(
    IEnumerable<ParcelGeometry> parcels,
    Polygon roadSurface)
{
    var contributions = new Dictionary<int, double>();

    foreach (var parcel in parcels)
    {
        if (parcel.Geometry.Intersects(roadSurface))
        {
            var overlap = parcel.Geometry.Intersection(roadSurface);
            contributions[parcel.ParcelId] = overlap.Area;
        }
    }

    return contributions;
}
}

```

7. SERVICE LAYER IMPLEMENTATION

7.1 GeometryService - Core Operations

```

using NetTopologySuite.Geometries;
using NetTopologySuite.Operation.Valid;
using NetTopologySuite.Simplify;

namespace RePlot.Services
{
    /// <summary>
    /// Centralized geometry operations service
    /// Used by all other services and UI layers
    /// </summary>
    public class GeometryService
    {
        private readonly GeometryFactory _factory;
        private readonly ReplotDbContext _context;

        public GeometryService(ReplotDbContext context)
        {
            var precision = new PrecisionModel(1000.0); // 1mm
            _factory = new GeometryFactory(precision, 6207); // Nepal TM
            _context = context;
        }
    }
}

```

#region Validation

```
/// <summary>
/// Validate geometry topology
/// </summary>
public (bool IsValid, string ErrorMessage, Coordinate? ErrorLocation)
    ValidateGeometry(Geometry geometry)
{
    if (geometry.IsValid)
        return (true, string.Empty, null);

    var validator = new IsValidOp(geometry);
    var error = validator.ValidationError;

    return (false, error.ErrorType.ToString(), error.Coordinate);
}
```

```
/// <summary>
/// Fix invalid geometry
/// </summary>
public Geometry FixGeometry(Geometry geometry)
{
    if (geometry.IsValid)
        return geometry;

    // Magic trick: buffer(0) fixes most topology errors
    return geometry.Buffer(0);
}
```

#endregion

#region Creation

```
public Polygon CreatePolygon(Coordinate[] coordinates)
{
    EnsureClosedRing(ref coordinates);
    var ring = _factory.CreateLinearRing(coordinates);
    return _factory.CreatePolygon(ring);
}

public LineString CreateLineString(Coordinate[] coordinates)
{
    return _factory.CreateLineString(coordinates);
}

public Point CreatePoint(double x, double y)
{
}
```

```

        return _factory.CreatePoint(new Coordinate(x, y));
    }

    private void EnsureClosedRing(ref Coordinate[] coordinates)
    {
        if (!coordinates[0].Equals2D(coordinates[coordinates.Length - 1]))
        {
            var closed = new Coordinate[coordinates.Length + 1];
            Array.Copy(coordinates, closed, coordinates.Length);
            closed[coordinates.Length] = new Coordinate(coordinates[0]);
            coordinates = closed;
        }
    }

#endregion

#region Measurement

    public double CalculateArea(Polygon polygon)
    {
        return polygon.Area;
    }

    public double CalculatePerimeter(Polygon polygon)
    {
        return polygon.Length;
    }

    public double CalculateDistance(Geometry geom1, Geometry geom2)
    {
        return geom1.Distance(geom2);
    }

    public Point GetCentroid(Polygon polygon)
    {
        return polygon.Centroid;
    }

    public Point GetLabelPoint(Polygon polygon)
    {
        // Interior point guaranteed to be inside (better for labels)
        return polygon.InteriorPoint;
    }

#endregion

#region Spatial Queries

```



```

public List<ParcelGeometry> FindParcelsInViewport(Envelope viewport)
{
    return _context.ParcelGeometries
        .Where(p =>
            p.BBoxMaxX >= viewport.MinX &&
            p.BBoxMinX <= viewport.MaxX &&
            p.BBoxMaxY >= viewport.MinY &&
            p.BBoxMinY <= viewport.MaxY)
        .ToList();
}

public List<ParcelGeometry> FindAdjacentParcels(int parcelId)
{
    var parcel = _context.ParcelGeometries.Find(parcelId);
    if (parcel == null)
        return new List<ParcelGeometry>();

    return _context.ParcelGeometries
        .Where(p => p.ParcelId != parcelId)
        .AsEnumerable() // Switch to client-side evaluation
        .Where(p => p.Geometry.Touches(parcel.Geometry))
        .ToList();
}

public List<ParcelGeometry> FindParcelsWithinDistance(
    Point location,
    double distanceMeters)
{
    var buffer = location.Buffer(distanceMeters);

    return _context.ParcelGeometries
        .AsEnumerable()
        .Where(p => p.Geometry.Intersects(buffer))
        .Where(p => p.Geometry.Distance(location) <= distanceMeters)
        .ToList();
}

public List<ParcelGeometry> FindOverlappingParcels(Polygon testParcel)
{
    var env = testParcel.EnvelopeInternal;

    return _context.ParcelGeometries
        .Where(p =>
            p.BBoxMaxX >= env.MinX &&
            p.BBoxMinX <= env.MaxX &&
            p.BBoxMaxY >= env.MinY &&

```

```

        p.BBoxMinY <= env.MaxY)
        .AsEnumerable()
        .Where(p => p.Geometry.Overlaps(testParcel))
        .ToList();
    }

#endregion

#region Operations

public Polygon MergeParcels(IEnumerable<Polygon> parcels)
{
    var collection = _factory.CreateGeometryCollection(parcels.ToArray());
    var merged = collection.Union();

    if (merged is Polygon polygon)
        return polygon;
    else
        throw new InvalidOperationException(
            "Parcels cannot be merged - they are not adjacent");
}

public List<Polygon> SplitParcel(Polygon parcel, LineString splitLine)
{
    var blade = splitLine.Buffer(0.001);
    var difference = parcel.Difference(blade);

    var result = new List<Polygon>();

    if (difference is MultiPolygon multi)
    {
        for (int i = 0; i < multi.NumGeometries; i++)
            result.Add((Polygon)multi.GetGeometryN(i));
    }
    else if (difference is Polygon single)
    {
        result.Add(single);
    }

    return result;
}

public Polygon ApplyBuffer(Polygon parcel, double distance)
{
    return (Polygon)parcel.Buffer(distance);
}

```

```

    public Polygon SimplifyGeometry(Polygon parcel, double tolerance)
    {
        return (Polygon)DouglasPeuckerSimplifier.Simplify(parcel, tolerance);
    }

    #endregion
}

```

7.2 Coordinate Conversion Service

```

namespace RePlot.Services
{
    /// <summary>
    /// Converts between screen coordinates and world coordinates (Nepal TM)
    /// Integrates with your existing ViewportEngine
    /// </summary>
    public class CoordinateConversionService
    {
        private readonly GeometryService _geometryService;

        public CoordinateConversionService(GeometryService geometryService)
        {
            _geometryService = geometryService;
        }

        /// <summary>
        /// Convert your IShape to NTS Geometry
        /// </summary>
        public Geometry ShapeToGeometry(IShape shape)
        {
            switch (shape)
            {
                case PolylineShape polyline when polyline.IsClosed:
                    return PolylineToPolygon(polyline);

                case PolylineShape polyline:
                    return PolylineToLineString(polyline);

                case LineShape line:
                    return LineToLineString(line);

                case RectangleShape rect:
                    return RectangleToPolygon(rect);

                case CircleShape circle:
                    return CircleToPolygon(circle);

                default:

```

```

        throw new NotSupportedException(
            $"Shape type {shape.GetType().Name} not supported");
    }
}

/// <summary>
/// Convert NTS Geometry back to IShape
/// </summary>
public IShape GeometryToShape(Geometry geometry, string layerName)
{
    switch (geometry)
    {
        case Polygon polygon:
            return PolygonToPolyline(polygon, layerName);

        case LineString lineString:
            return LineStringToPolyline(lineString, layerName);

        case Point point:
            // Return as tiny circle for visualization
            return new CircleShape
            {
                Center = new PointD(point.X, point.Y),
                Radius = 1.0,
                LayerName = layerName
            };

        default:
            throw new NotSupportedException(
                $"Geometry type {geometry.GeometryType} not supported");
    }
}

private Polygon PolylineToPolygon(PolylineShape polyline)
{
    var coords = polyline.Vertices
        .Select(v => new Coordinate(v.X, v.Y))
        .ToArray();

    return _geometryService.CreatePolygon(coords);
}

private LineString PolylineToLineString(PolylineShape polyline)
{
    var coords = polyline.Vertices
        .Select(v => new Coordinate(v.X, v.Y))
        .ToArray();

```

```

        return _geometryService.CreateLineString(coords);
    }

    private LineString LineToLineString(LineShape line)
    {
        var coords = new[]
        {
            new Coordinate(line.Start.X, line.Start.Y),
            new Coordinate(line.End.X, line.End.Y)
        };

        return _geometryService.CreateLineString(coords);
    }

    private Polygon RectangleToPolygon(RectangleShape rect)
    {
        var coords = new[]
        {
            new Coordinate(rect.Start.X, rect.Start.Y),
            new Coordinate(rect.End.X, rect.Start.Y),
            new Coordinate(rect.End.X, rect.End.Y),
            new Coordinate(rect.Start.X, rect.End.Y),
            new Coordinate(rect.Start.X, rect.Start.Y)
        };

        return _geometryService.CreatePolygon(coords);
    }

    private Polygon CircleToPolygon(CircleShape circle)
    {
        var point = _geometryService.CreatePoint(circle.Center.X, circle.Center.Y);
        return (Polygon)point.Buffer(circle.Radius);
    }

    private PolylineShape PolygonToPolyline(Polygon polygon, string layerName)
    {
        var vertices = polygon.ExteriorRing.Coordinates
            .Select(c => new PointD(c.X, c.Y))
            .ToList();

        return new PolylineShape
        {
            Vertices = vertices,
            IsClosed = true,
            LayerName = layerName
        };
    }

```

```

    }

    private PolylineShape LineStringToPolyline(LineString lineString, string layerName)
    {
        var vertices = lineString.Coordinates
            .Select(c => new PointD(c.X, c.Y))
            .ToList();

        return new PolylineShape
        {
            Vertices = vertices,
            IsClosed = lineString.IsClosed,
            LayerName = layerName
        };
    }
}

```

8. PRACTICAL CODE EXAMPLES

8.1 Complete Parcel Workflow

```

public class ParcelWorkflowExample
{
    private readonly GeometryService _geometryService;
    private readonly CoordinateConversionService _conversionService;
    private readonly ReplotDbContext _context;

    public ParcelWorkflowExample(
        GeometryService geometryService,
        CoordinateConversionService conversionService,
        ReplotDbContext context)
    {
        _geometryService = geometryService;
        _conversionService = conversionService;
        _context = context;
    }

    /// <summary>
    /// Complete workflow: Create parcel from drawing → Save to DB
    /// </summary>
    public async Task<ParcelGeometry> CreateAndSaveParcel(
        PolylineShape drawnParcel,
        int parcelId,
        int layerId)
    {
        // Step 1: Convert IShape to NTS Geometry
        var polygon = (Polygon)_conversionService.ShapeToGeometry(drawnParcel);
    }
}

```

```

// Step 2: Validate
var (isValid, errorMsg, errorCoord) = _geometryService.ValidateGeometry(polygon);
if (!isValid)
{
    throw new InvalidOperationException(
        $"Invalid parcel: {errorMsg} at ({errorCoord?.X}, {errorCoord?.Y})");
}

// Step 3: Create entity
var parcelGeom = new ParcelGeometry
{
    ParcelId = parcelId,
    LayerId = layerId,
    Geometry = polygon
};
parcelGeom.UpdateBoundingBox();

// Step 4: Save to database
_context.ParcelGeometries.Add(parcelGeom);
await _context.SaveChangesAsync();

return parcelGeom;
}

/// <summary>
/// Merge two adjacent parcels
/// </summary>
public async Task<ParcelGeometry> MergeTwoParcels(int parcel1Id, int parcel2Id)
{
    // Load from database
    var parcel1 = await _context.ParcelGeometries.FindAsync(parcel1Id);
    var parcel2 = await _context.ParcelGeometries.FindAsync(parcel2Id);

    if (parcel1 == null || parcel2 == null)
        throw new ArgumentException("Parcel not found");

    // Check if they touch
    if (!parcel1.Geometry.Touches(parcel2.Geometry))
        throw new InvalidOperationException("Parcels are not adjacent");

    // Merge
    var merged = _geometryService.MergeParcels(new[]
    {
        parcel1.Geometry,
        parcel2.Geometry
    });
}

```

```

// Create new parcel
var newParcel = new ParcelGeometry
{
    ParcelId = GenerateNewParcelId(),
    LayerId = parcel1.LayerId,
    Geometry = merged
};
newParcel.UpdateBoundingBox();

// Update database
_context.ParcelGeometries.Remove(parcel1);
_context.ParcelGeometries.Remove(parcel2);
_context.ParcelGeometries.Add(newParcel);
await _context.SaveChangesAsync();

return newParcel;
}

/// <summary>
/// Calculate road contribution for land pooling
/// </summary>
public async Task<Dictionary<int, double>> CalculateRoadContributions(
    LineString roadCenterline,
    double roadWidth)
{
    // Create road surface
    var roadSurface = (Polygon)roadCenterline.Buffer(roadWidth / 2.0);

    // Find all intersecting parcels
    var env = roadSurface.EnvelopeInternal;
    var affectedParcels = _context.ParcelGeometries
        .Where(p =>
            p.BBoxMaxX >= env.MinX &&
            p.BBoxMinX <= env.MaxX &&
            p.BBoxMaxY >= env.MinY &&
            p.BBoxMinY <= env.MaxY)
        .ToList();

    // Calculate contributions
    var contributions = new Dictionary<int, double>();

    foreach (var parcel in affectedParcels)
    {
        if (parcel.Geometry.Intersects(roadSurface))
        {
            var overlap = parcel.Geometry.Intersection(roadSurface);
            contributions[parcel.ParcelId] = overlap.Area;
        }
    }
}

```



```

    }
}

return contributions;
}

/// <summary>
/// Apply setback and calculate buildable area
/// </summary>
public double CalculateBuildableArea(int parcelId, double setbackDistance)
{
    var parcel = _context.ParcelGeometries.Find(parcelId);
    if (parcel == null)
        return 0;

    var buildable = _geometryService.ApplyBuffer(parcel.Geometry, -setbackDistance);

    return buildable.Area;
}

private int GenerateNewParcelId()
{
    // Implementation depends on your ID generation strategy
    return _context.ParcelGeometries.Max(p => p.ParcelId) + 1;
}
}

```

8.2 Viewport Rendering Integration

```

public class CanvasRenderingWithNTS
{
    private readonly GeometryService _geometryService;
    private readonly STRtree<ParcelGeometry> _spatialIndex;

    public CanvasRenderingWithNTS(GeometryService geometryService)
    {
        _geometryService = geometryService;
        _spatialIndex = new STRtree<ParcelGeometry>();
    }

    /// <summary>
    /// Build spatial index from database
    /// Call this once after loading project
    /// </summary>
    public void BuildSpatialIndex(List<ParcelGeometry> allParcels)
    {
        foreach (var parcel in allParcels)
        {
            var env = parcel.Geometry.EnvelopeInternal;

```

```

        _spatialIndex.Insert(env, parcel);
    }

    _spatialIndex.Build(); // CRITICAL: Make queries fast!
}

/// <summary>
/// Get parcels to render in current viewport
/// Integrates with your OptimizedDeferredRenderer
/// </summary>
public List<ParcelGeometry> GetVisibleParcels(RectangleD viewportBounds)
{
    // Convert viewport to NTS Envelope
    var envelope = new Envelope(
        viewportBounds.X,
        viewportBounds.X + viewportBounds.Width,
        viewportBounds.Y,
        viewportBounds.Y + viewportBounds.Height
    );

    // Query R-Tree (fast!)
    var candidates = _spatialIndex.Query(envelope);

    // Precise filtering (optional - depends on needs)
    var viewportPoly = CreateViewportPolygon(viewportBounds);
    var visible = candidates
        .Where(p => p.Geometry.Intersects(viewportPoly))
        .ToList();

    return visible;
}

/// <summary>
/// Render parcel on canvas
/// Converts NTS Polygon to screen coordinates
/// </summary>
public void RenderParcel(
    Graphics g,
    ParcelGeometry parcel,
    Func<PointD, PointD> worldToScreen,
    bool isSelected)
{
    var coords = parcel.Geometry.ExteriorRing.Coordinates;
    var screenPoints = coords
        .Select(c => worldToScreen(new PointD(c.X, c.Y)))
        .Select(p => new PointF((float)p.X, (float)p.Y))
        .ToArray();

```

```

// Fill
using (var brush = new SolidBrush(
    isSelected ? Color.FromArgb(100, 0, 120, 215) : Color.FromArgb(50, 200, 200, 200)))
{
    g.FillPolygon(brush, screenPoints);
}

// Border
using (var pen = new Pen(isSelected ? Color.Blue : Color.Black, isSelected ? 2f : 1f))
{
    g.DrawPolygon(pen, screenPoints);
}

// Label (area)
var centroid = parcel.Geometry.InteriorPoint;
var labelPos = worldToScreen(new PointD(centroid.X, centroid.Y));
var label = $"{parcel.AreaSqm:F2} m²";

using (var font = new Font("Arial", 10))
using (var brush = new SolidBrush(Color.Black))
{
    g.DrawString(label, font, brush, (float)labelPos.X, (float)labelPos.Y);
}
}

private Polygon CreateViewportPolygon(RectangleD viewport)
{
    var coords = new[]
    {
        new Coordinate(viewport.X, viewport.Y),
        new Coordinate(viewport.X + viewport.Width, viewport.Y),
        new Coordinate(viewport.X + viewport.Width, viewport.Y + viewport.Height),
        new Coordinate(viewport.X, viewport.Y + viewport.Height),
        new Coordinate(viewport.X, viewport.Y)
    };

    return _geometryService.CreatePolygon(coords);
}
}

```

8.3 Import/Export Integration

```

public class ImportExportService
{
    private readonly GeometryService _geometryService;

    public ImportExportService(GeometryService geometryService)
    {

```

```

    _geometryService = geometryService;
}

/// <summary>
/// Import parcels from GeoJSON (QGIS, web maps)
/// </summary>
public List<ParcelGeometry> ImportFromGeoJSON(string geoJsonFilePath)
{
    var geoJson = File.ReadAllText(geoJsonFilePath);
    var reader = new GeoJsonReader();
    var featureCollection = reader.Read<FeatureCollection>(geoJson);

    var parcels = new List<ParcelGeometry>();

    foreach (var feature in featureCollection)
    {
        if (feature.Geometry is Polygon polygon)
        {
            var parcel = new ParcelGeometry
            {
                Geometry = polygon,
                ParcelId = Convert.ToInt32(feature.Attributes["ParcelId"]),
                LayerId = Convert.ToInt32(feature.Attributes["LayerId"])
            };
            parcel.UpdateBoundingBox();
            parcels.Add(parcel);
        }
    }

    return parcels;
}

/// <summary>
/// Export parcels to GeoJSON
/// </summary>
public void ExportToGeoJSON(List<ParcelGeometry> parcels, string outputPath)
{
    var features = parcels.Select(p => new Feature
    {
        Geometry = p.Geometry,
        Attributes = new AttributesTable
        {
            { "ParcelId", p.ParcelId },
            { "LayerId", p.LayerId },
            { "Area", p.AreaSqm },
            { "Perimeter", p.Perimeter }
        }
    });
}

```

```

    }).ToList();

    var featureCollection = new FeatureCollection(features);

    var writer = new GeoJsonWriter();
    var geoJson = writer.Write(featureCollection);
    File.WriteAllText(outputPath, geoJson);
}

/// <summary>
/// Import from WKT text file (CSV with WKT column)
/// </summary>
public List<ParcelGeometry> ImportFromWKT(string csvFilePath)
{
    var reader = new WKTReader();
    var parcels = new List<ParcelGeometry>();

    var lines = File.ReadAllLines(csvFilePath).Skip(1); // Skip header

    foreach (var line in lines)
    {
        var parts = line.Split(',');
        var parcelId = int.Parse(parts[0]);
        var layerId = int.Parse(parts[1]);
        var wkt = parts[2];

        var geometry = reader.Read(wkt);

        if (geometry is Polygon polygon)
        {
            var parcel = new ParcelGeometry
            {
                ParcelId = parcelId,
                LayerId = layerId,
                Geometry = polygon
            };
            parcel.UpdateBoundingBox();
            parcels.Add(parcel);
        }
    }

    return parcels;
}

/// <summary>
/// Export to AutoCAD DXF (requires additional library)
/// </summary>

```

```

public void ExportToDXF(List<ParcelGeometry> parcels, string outputPath)
{
    // This requires netDxf library (separate NuGet package)
    // Placeholder for integration point

    // Convert NTS Polygon → DXF Polyline entities
    // Group by layer
    // Write to file

    throw new NotImplementedException("DXF export requires netDxf library");
}
}

```

9. PERFORMANCE OPTIMIZATION

9.1 Spatial Index Best Practices

```

public class PerformanceOptimizedSpatialQuery
{
    private readonly STRtree<ParcelGeometry> _index;
    private readonly ReplotDbContext _context;

    public PerformanceOptimizedSpatialQuery(ReplotDbContext context)
    {
        _context = context;
        _index = new STRtree<ParcelGeometry>();
        RebuildIndex();
    }

    /// <summary>
    /// Rebuild index after bulk changes
    /// </summary>
    public void RebuildIndex()
    {
        var allParcels = _context.ParcelGeometries.ToList();

        foreach (var parcel in allParcels)
        {
            _index.Insert(parcel.Geometry.EnvelopeInternal, parcel);
        }

        _index.Build(); // CRITICAL: Must call after all inserts!
    }

    /// <summary>
    /// Incremental update (faster than full rebuild)
    /// </summary>
    public void UpdateParcelInIndex(ParcelGeometry parcel)
    {

```

```

        // STRtree doesn't support removal - need to rebuild
        // For frequent updates, use Quadtree instead
        RebuildIndex();
    }

    /// <summary>
    /// Two-stage filtering for best performance
    /// </summary>
    public List<ParcelGeometry> FindIntersectingParcels(Polygon testParcel)
    {
        // Stage 1: Fast bounding box query (R-Tree)
        var env = testParcel.EnvelopeInternal;
        var candidates = _index.Query(env);

        // Stage 2: Precise geometry intersection (slower)
        var result = candidates
            .Where(p => p.Geometry.Intersects(testParcel))
            .ToList();

        return result;
    }
}

```

9.2 Database Query Optimization

```

public class OptimizedDatabaseQueries
{
    private readonly ReplotDbContext _context;

    public OptimizedDatabaseQueries(ReplotDbContext context)
    {
        _context = context;
    }

    /// <summary>
    /// Use bounding box for initial filtering (FAST)
    /// Then precise geometry check (SLOWER)
    /// </summary>
    public List<ParcelGeometry> FindParcelsInArea(Envelope searchArea)
    {
        // Stage 1: Database query using indexed bounding box columns
        var candidates = _context.ParcelGeometries
            .Where(p =>
                p.BBoxMaxX >= searchArea.MinX &&
                p.BBoxMinX <= searchArea.MaxX &&
                p.BBoxMaxY >= searchArea.MinY &&
                p.BBoxMinY <= searchArea.MaxY)
            .ToList(); // Bring to memory
    }
}

```

```

// Stage 2: Precise filtering in memory
var factory = NtsGeometryServices.Instance.CreateGeometryFactory();
var searchPoly = factory.ToGeometry(searchArea);

var result = candidates
    .Where(p => p.Geometry.Intersects(searchPoly))
    .ToList();

return result;
}

/// <summary>
/// Batch loading for large datasets
/// </summary>
public async Task<List<ParcelGeometry>> LoadParcelsInBatchesAsync(int batchSize =
1000)
{
    var totalCount = await _context.ParcelGeometries.CountAsync();
    var allParcels = new List<ParcelGeometry>(totalCount);

    for (int skip = 0; skip < totalCount; skip += batchSize)
    {
        var batch = await _context.ParcelGeometries
            .OrderBy(p => p.GeometryId)
            .Skip(skip)
            .Take(batchSize)
            .ToListAsync();

        allParcels.AddRange(batch);

        // Report progress
        var progress = (skip + batch.Count) * 100 / totalCount;
        Console.WriteLine($"Loading parcels: {progress}%");
    }

    return allParcels;
}
}

```

9.3 Geometry Simplification

```

public class GeometrySimplificationService
{
    /// <summary>
    /// Simplify for display at different zoom levels
    /// </summary>
    public Polygon SimplifyForZoomLevel(Polygon parcel, double zoomLevel)
    {
        // Calculate tolerance based on zoom
    }
}

```



```

// At low zoom (zoomed out), higher tolerance = fewer vertices
double tolerance = 1.0 / zoomLevel;

var simplified = DouglasPeuckerSimplifier.Simplify(parcel, tolerance);

return (Polygon)simplified;
}

/// <summary>
/// Create Level-of-Detail (LOD) versions
/// </summary>
public Dictionary<string, Polygon> CreateLODVersions(Polygon parcel)
{
    return new Dictionary<string, Polygon>
    {
        ["High"] = parcel, // Original
        ["Medium"] = (Polygon)DouglasPeuckerSimplifier.Simplify(parcel, 0.5),
        ["Low"] = (Polygon)DouglasPeuckerSimplifier.Simplify(parcel, 2.0),
        ["VeryLow"] = (Polygon)parcel.ConvexHull // Just bounding polygon
    };
}
}

```

10. COMMON PITFALLS AND SOLUTIONS

10.1 Problem: Self-Intersecting Polygons

```

// ❌ WRONG: Create bowtie polygon
var coords = new[]
{
    new Coordinate(0, 0),
    new Coordinate(100, 100), // Crosses back!
    new Coordinate(100, 0),
    new Coordinate(0, 100),
    new Coordinate(0, 0)
};
var invalid = factory.CreatePolygon(factory.CreateLinearRing(coords));
// invalid.IsValid = false!

// ✅ SOLUTION 1: Validate before save
if (!polygon.IsValid)
{
    var validator = new IsValidOp(polygon);
    var error = validator.ValidationErrors;
    throw new InvalidOperationException(
        $"Self-intersection at ({error.Coordinate.X}, {error.Coordinate.Y})");
}

// ✅ SOLUTION 2: Auto-fix

```

```
var fixed = polygon.Buffer(0); // Magic trick!
```

10.2 Problem: Unclosed Rings

```
// ❌ WRONG: Forget to close
```

```
var coords = new[]
```

```
{
    new Coordinate(0, 0),
    new Coordinate(100, 0),
    new Coordinate(100, 100),
    new Coordinate(0, 100)
    // Missing: new Coordinate(0, 0)
};
```

```
// Throws: ArgumentException - Ring not closed!
```

```
// ✅ SOLUTION: Helper method
```

```
private Coordinate[] EnsureClosedRing(Coordinate[] coords)
```

```
{
    if (!coords[0].Equals2D(coords[coords.Length - 1]))
    {
        var closed = new Coordinate[coords.Length + 1];
        Array.Copy(coords, closed, coords.Length);
        closed[coords.Length] = new Coordinate(coords[0]);
        return closed;
    }
    return coords;
}
```

10.3 Problem: Precision Loss

```
// ❌ WRONG: Default factory loses precision
```

```
var factory = new GeometryFactory(); // Floating precision, no SRID
```

```
var point = factory.CreatePoint(new Coordinate(535123.456789, 3012456.789012));
```

```
// Stored as full double, but no coordinate system reference!
```

```
// ✅ SOLUTION: Set precision model and SRID
```

```
var precisionModel = new PrecisionModel(1000.0); // 1mm accuracy
```

```
var factory = new GeometryFactory(precisionModel, 6207); // Nepal TM
```

10.4 Problem: Wrong SRID

```
// ❌ WRONG: Mix coordinate systems
```

```
var wgs84Point = factory.CreatePoint(new Coordinate(85.3240, 27.7172)); // Lat/Lon
```

```
var nepalTmParcel = GetParcelFromDatabase(); // Nepal TM (meters)
```

```
var distance = wgs84Point.Distance(nepalTmParcel.Geometry.Centroid);
```

```
// Distance is NONSENSE! Mixing degrees and meters!
```

```
// ✅ SOLUTION: Always use same coordinate system
```

```
// Project WGS84 to Nepal TM BEFORE calculations
```

```
// Or use ProjNet library for coordinate transformation
```

10.5 Problem: Forgetting to Update Bounding Box

```
// ❌ WRONG: Modify geometry but don't update bbox
```

```
parcel.Geometry = parcel.Geometry.Buffer(-5.0) as Polygon;  
context.SaveChanges();  
// BBox columns still have OLD values! Spatial queries fail!
```

```
// ✅ SOLUTION: Always call UpdateBoundingBox()  
parcel.Geometry = parcel.Geometry.Buffer(-5.0) as Polygon;  
parcel.UpdateBoundingBox(); // ← CRITICAL!  
context.SaveChanges();
```

10.6 Problem: STRtree Not Built

```
// ❌ WRONG: Query without building  
var index = new STRtree<ParcelGeometry>();  
foreach (var parcel in parcels)  
    index.Insert(parcel.Geometry.EnvelopeInternal, parcel);  
  
var result = index.Query(envelope); // SLOW! Not optimized!
```

```
// ✅ SOLUTION: Call Build() after all inserts  
var index = new STRtree<ParcelGeometry>();  
foreach (var parcel in parcels)  
    index.Insert(parcel.Geometry.EnvelopeInternal, parcel);
```

```
index.Build(); // ← CRITICAL! Makes queries fast!  
var result = index.Query(envelope); // Now FAST!
```

10.7 Problem: Holes Outside Shell

```
// ❌ WRONG: Hole completely outside parcel boundary  
var shell = CreateRing(parcelCoords);  
var hole = CreateRing(buildingCoords); // Oops! Building is 1km away!  
var invalid = factory.CreatePolygon(shell, new[] { hole });  
// invalid.IsValid = false - "Hole lies outside shell"
```

```
// ✅ SOLUTION: Validate hole is within shell  
if (!shellPolygon.Contains(holePolygon))  
    throw new InvalidOperationException("Hole must be inside parcel boundary");
```

SUMMARY & NEXT STEPS

What You've Learned

1. ✅ **NTS Basics:** Geometry types, creation, properties
2. ✅ **Database Integration:** Entity Framework + SQLite spatial
3. ✅ **Service Architecture:** GeometryService, conversion layers
4. ✅ **Parcel Operations:** Create, merge, split, buffer, validate
5. ✅ **Performance:** Spatial indexing, query optimization
6. ✅ **Common Pitfalls:** Self-intersections, precision, SRID issues

Implementation Checklist

- [] Install NuGet packages (NTS + EF Core Spatial)
- [] Create GeometryService with factory

- [] Update DbContext with NTS support
- [] Add geometry columns to database models
- [] Implement CoordinateConversionService
- [] Build spatial index (STRtree)
- [] Integrate with viewport rendering
- [] Add validation before saving
- [] Test with sample parcel data
- [] Implement import/export (GeoJSON, WKT)

Where to Start

1. **Phase 1:** Basic setup
 - o Add NuGet packages
 - o Create GeometryService
 - o Test creating simple polygons
2. **Phase 2:** Database integration
 - o Update models with NTS types
 - o Configure DbContext
 - o Run migrations
 - o Save/load one parcel
3. **Phase 3:** Operations
 - o Implement merge
 - o Implement split
 - o Validate topologies
 - o Test with real data
4. **Phase 4:** Performance
 - o Build spatial index
 - o Optimize viewport queries
 - o Add bounding box filtering
5. **Phase 5:** Integration
 - o Connect to drawing canvas
 - o Import from files
 - o Export to GeoJSON

ADDITIONAL RESOURCES

Official Documentation:

- [NetTopology Suite GitHub](#)
- [NTS Wiki](#)

Coordinate Systems:

- [EPSG:6207 - Nepal TM](#)

Related Libraries:

- ProjNet (coordinate transformations)
 - netDxf (AutoCAD export)
 - SharpKml (KML/KMZ export)
-

This guide gives you EVERYTHING needed to implement NTS in RePlot. Start with Phase 1, test each phase, and build incrementally. You now have the knowledge to master parcel geometry operations!